

Data Warehouses Project Report

Student name: Flavia Medrea

E-mail: flavia.medrea01@e-uvr.ro

Table of Contents

1. Introduction	2
2. System Architecture	2
3. Technology Stack	3
4. System Requirements and Deployment	4
5. Conclusions	5

1. Introduction

This project report describes the development of a complete end-to-end financial data warehouse for a company called Acme Ltd. The main objective of the system is to collect different types of market data from multiple third-party vendors, organize and store the data in a structured way, and use it to assess asset risks and provide financial recommendations to customers. To achieve these goals, the system combines real-time data streaming, an append-only temporal database design, and modern artificial intelligence technologies within a single integrated platform.

2. System Architecture

The platform is built using a decoupled multi-tier architecture, which separates the main system components into different layers. This approach improves scalability, flexibility, and maintainability by allowing each layer to operate independently.

Presentation Tier (Frontend)

The frontend is developed using Streamlit and provides a user-friendly web interface. It communicates with the backend through HTTPX requests and displays information such as portfolio valuations, historical financial data, and risk metrics. Interactive visualizations are created using Plotly charts to help users better understand the data.

Application Tier (Middleware/API)

This layer is implemented with FastAPI and contains the core business logic of the system. It manages data ingestion processes, performs data processing tasks, and exposes REST API endpoints. Apache Spark is integrated into this layer to perform large-scale aggregations on financial data. In addition, it includes a Model Context Protocol (MCP) server, which allows a Large Language Model (LLM) to safely interact with the database through predefined tools and functions.

Data Tier (Storage)

The storage layer uses MongoDB, a flexible NoSQL database that can store complex document structures. This design is well suited for financial data because different asset types may have different attributes. As a result, the database can efficiently store polymorphic financial assets without allocating unnecessary space for unused fields.

3. Technology Stack

The technologies used in this project were selected based on the system requirements for scalability, flexibility, and real-time data processing.

Technology	Purpose	Reason for Selection
MongoDB	Database	Supports flexible document structures, making it suitable for storing different types of financial assets with varying attributes.
FastAPI	Backend API	Provides high performance and asynchronous processing, allowing the system to handle multiple data streams efficiently.
Motor	Database Driver	Enables asynchronous communication between FastAPI and MongoDB, improving application responsiveness.
Streamlit	Frontend Dashboard	Simplifies the development of interactive data-driven web applications using Python.
Plotly	Data Visualization	Creates interactive charts for displaying financial data and portfolio analytics.
Apache Spark	Data Analytics	Provides efficient processing of large datasets and supports analytical workloads such as data aggregation and predictive analysis.
MCP	LLM Integration	Allows the AI assistant to access platform data through controlled tools and functions, improving the accuracy of generated insights.

4. System Requirements and Deployment

Hardware Requirements

The platform can be executed on a standard personal computer with the following minimum specifications:

- Dual-core processor or higher
- 8 GB RAM
- 5 GB available disk space
- Internet connection for retrieving financial market data

Software Requirements

The following software must be installed before running the application:

- Python 3.10 or later
- MongoDB Community Server
- MongoDB Compass (optional, but recommended for database inspection)
- An IDE or code editor (e.g., Visual Studio Code or PyCharm)

Environment Setup

The project can be executed using either a global Python installation or an isolated virtual environment. The latter is recommended to avoid dependency conflicts.

```
python -m venv venv
```

The virtual environment can then be activated:

```
# Windows
.\venv\Scripts\Activate.ps1
```

```
# macOS/Linux
source venv/bin/activate
```

The required dependencies are installed using:

```
pip install fastapi uvicorn motor httpx streamlit plotly pandas
```

Running the Application

After MongoDB has been started, inside the project folder, the backend API can be launched using:

```
python -m uvicorn app.main:app --reload
```

In a separate terminal window, the Streamlit user interface can be started:

```
cd app  
streamlit run interface.py
```

Once both services are running, the platform is accessible through the Streamlit web interface, which communicates with the FastAPI backend and MongoDB database.

For understanding how to use the app check the demo video.

5. Conclusions

This project successfully delivers a working financial data warehouse for Acme Ltd that proves why NoSQL databases are a great fit for modern, fast-moving data. By using MongoDB instead of a traditional relational setup, the platform easily handles different asset classes and future proofing without needing a complex schema change. The append-only, bitemporal model works perfectly to ensure data history is never lost or overwritten, allowing users to accurately pull historical portfolio valuations and audit exactly where market data came from. Finally, splitting up the asynchronous FastAPI backend from the interactive Streamlit UI keeps the app running fast, while the built-in MCP server makes it incredibly simple to connect an AI assistant that can summarize data trends using facts, not generic guesses.